

# ComputeModule — Correct GEMM & ALU Implementation

This guide corrects the previous version. It uses only constants that already exist in the project ( `vta_config.h` ), introduces only what `vta.cc` itself declares, and explains every number instead of using magic values.

## 1. What the Previous Guide Got Wrong — and Why

| Error in previous guide                                                                                                                                                          | What is actually correct                                                                                                                                                                                                                                                                                     |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Used magic numbers like <code>2048</code> , <code>512</code> , <code>16</code> directly in the arrays                                                                            | These come from VTA constants. The project has <code>vta_config.h</code> which defines the width constants. Use those — or at minimum name the constants and explain where each comes from.                                                                                                                  |
| Invented new arrays ( <code>uop_mem</code> , <code>acc_mem</code> , <code>inp_mem</code> , <code>wgt_mem</code> , <code>out_mem</code> ) without explaining where they come from | In <code>vta.cc</code> , <code>compute()</code> declares <code>static uop_mem</code> and <code>static acc_mem</code> inside itself. <code>inp_mem</code> , <code>wgt_mem</code> , <code>out_mem</code> are passed as parameters from the top-level <code>vta()</code> . We must mirror this exact structure. |
| Added things that the professor said are not in <code>.h</code>                                                                                                                  | The new static arrays go in <b><code>compute_module.cpp</code> only</b> , just as <code>vta.cc</code> 's <code>compute()</code> declares them locally. No <code>.h</code> changes.                                                                                                                           |
| Used <code>0x7FF</code> and <code>0x3FF</code> without explanation                                                                                                               | <code>vta_config.h</code> already defines <code>UOP_DST_WIDTH = 11</code> , <code>UOP_SRC_WIDTH = 11</code> , <code>UOP_WGT_WIDTH = 10</code> . The masks must be computed from these.                                                                                                                       |

## 2. Where Constants Come From

### 2.1 Constants already in `vta_config.h` (the project already has these)

```
namespace vta_config {
    // Bit widths of the index fields inside a 32-bit micro-op word:
    constexpr int UOP_DST_WIDTH = 11;    // acc tile index = 11 bits → max 2047
    constexpr int UOP_SRC_WIDTH = 11;    // inp / acc-src tile index = 11 bits → max 2047
    constexpr int UOP_WGT_WIDTH = 10;    // wgt tile index = 10 bits → max 1023

    // Loop field widths (in the instruction word, not the buffer):
    constexpr int UOP_BGN_WIDTH = 13;
    constexpr int UOP_END_WIDTH = 14;
    constexpr int ITER_OUT_WIDTH = 14;
    constexpr int ITER_IN_WIDTH = 14;

    // ALU opcodes (already an enum in vta_config.h):
    enum class AluOpcode : uint8_t { MIN=0, MAX=1, ADD=2, SHR=3, MUL=4 };
}
```

### 2.2 Constants from VTA `hw_spec.h` (NOT yet in this project — must be defined locally)

`vta.cc` uses these constants which come from `hw_spec.h` (the TVM hardware specification). Since the simulator does not include `hw_spec.h`, they must be defined manually in `compute_module.cpp`:

| VTA constant (from <code>hw_spec.h</code> ) | Meaning                                            | Default value | Derived from                                            |
|---------------------------------------------|----------------------------------------------------|---------------|---------------------------------------------------------|
| <code>VTA_BATCH</code>                      | Batch size (# of input vectors per tile)           | 1             | Hardware parameter fixed at 1 in most VTA configs       |
| <code>VTA_BLOCK_IN</code>                   | Input tile width (# of input channels per tile)    | 16            | Hardware parameter                                      |
| <code>VTA_BLOCK_OUT</code>                  | Output tile height (# of output channels per tile) | 16            | Hardware parameter                                      |
| <code>VTA_UOP_BUFF_DEPTH</code>             | Number of entries in micro-op SRAM                 | 2048          | $1 \ll 11$ (max index from <code>UOP_DST_WIDTH</code> ) |
| <code>VTA_ACC_BUFF_DEPTH</code>             | Number of tiles in accumulator SRAM                | 2048          | $1 \ll 11$ (max index from <code>UOP_DST_WIDTH</code> ) |
| <code>VTA_INP_BUFF_DEPTH</code>             | Number of input tiles in SRAM                      | 2048          | $1 \ll 11$ (max index from <code>UOP_SRC_WIDTH</code> ) |
| <code>VTA_WGT_BUFF_DEPTH</code>             | Number of weight tiles in SRAM                     | 1024          | $1 \ll 10$ (max index from <code>UOP_WGT_WIDTH</code> ) |

#### How to derive the buffer depths from `vta_config.h`:

A field of width N bits can hold indices  $0 \dots 2^N - 1$ . So a buffer indexed by that field needs  $2^N$  entries. Using `vta_config::UOP_DST_WIDTH = 11 :  $1 \ll 11 = 2048$` . This is how `vta.cc`'s `VTA_ACC_BUFF_DEPTH` and `VTA_UOP_BUFF_DEPTH` are derived.

### 3. Bitmask Explained Properly

In `vta.cc`, Xilinx HLS provides `ap_uint.range(hi, lo)` to extract a bitfield. Standard C++ has no such method — we must use shift (`>>`) and mask (`&`). The mask is always `(1 << WIDTH) - 1`:

```
// GEMM micro-op bit layout (32 bits total):
//
// Bit 31          22  21          11  10          0
//
// |   wgt_base   |   src_base   |   dst_base   |
// | (10 bits)    | (11 bits)    | (11 bits)    |
// | UOP_WGT_WIDTH | UOP_SRC_WIDTH | UOP_DST_WIDTH |
//
//
// To extract each field (C++ replacement for HLS uop.range(hi, lo)):

constexpr uint32_t DST_MASK = (1u << vta_config::UOP_DST_WIDTH) - 1; // = 0x7FF (2047)
constexpr uint32_t SRC_MASK = (1u << vta_config::UOP_SRC_WIDTH) - 1; // = 0x7FF (2047)
constexpr uint32_t WGT_MASK = (1u << vta_config::UOP_WGT_WIDTH) - 1; // = 0x3FF (1023)

int dst_base = (int)((uop >> 0) & DST_MASK); // bits [10:0]
int src_base = (int)((uop >> vta_config::UOP_DST_WIDTH) & SRC_MASK); // bits [21:11]
int wgt_base = (int)((uop >> (vta_config::UOP_DST_WIDTH + vta_config::UOP_SRC_WIDTH)) & WGT_MASK); // bi
```

**Why these exact bit positions?** The three fields are packed consecutively: dst occupies the lowest bits (width=11), then src (width=11) right above it, then wgt (width=10) on top. So src starts at bit 11 (= `UOP_DST_WIDTH`), and wgt starts at bit 22 (= `UOP_DST_WIDTH + UOP_SRC_WIDTH`).

### 4. Memory Arrays — Only What `vta.cc` Actually Declares

In `vta.cc compute()` (the exact lines, copied):

```
static uop_T uop_mem[VTA_UOP_BUFF_DEPTH]; // micro-op SRAM — declared STATIC inside compute
static bus_T acc_mem[VTA_ACC_BUFF_DEPTH][ACC_MAT_AXI_RATIO]; // accumulator — STATIC inside compute

// inp_mem, wgt_mem, out_mem are PARAMETERS passed in from vta() — not declared here
```

In SystemC `compute_module.cpp` — the exact parallel:

`uop_mem` and `acc_mem` are private to Compute — they go in **`compute_module.cpp` only** as static file-scope variables (same effect as HLS `static` inside a function: values persist between calls).

`inp_mem`, `wgt_mem`, `out_mem` are shared between Load, Compute, and Store — they need to be accessible

from all three modules. For now, we define them in **compute\_module.cpp** and note that Load and Store would need a way to access them (via `extern` or a shared header, which is a separate design decision).

```
// — In compute_module.cpp, after the #include lines —————
// Mirror of vta.cc compute()'s static declarations – no .h changes needed

// From vta_config.h: UOP_DST_WIDTH=11, UOP_SRC_WIDTH=11, UOP_WGT_WIDTH=10
// Buffer depths derived: 1<<11 = 2048, 1<<10 = 1024

// VTA hardware block sizes (from VTA hw_spec.h – define here since simulator lacks hw_spec.h)
constexpr int VTA_BATCH      = 1; // batch dimension (always 1 in standard VTA)
constexpr int VTA_BLOCK_IN   = 16; // input tile width (BLOCK_IN input channels per tile)
constexpr int VTA_BLOCK_OUT  = 16; // output tile height (BLOCK_OUT output channels per tile)

// Buffer depths (derived from vta_config.h bit-widths)
constexpr int UOP_BUFF_DEPTH = 1 << vta_config::UOP_DST_WIDTH; // 2048 uop entries
constexpr int ACC_BUFF_DEPTH = 1 << vta_config::UOP_DST_WIDTH; // 2048 acc tiles
constexpr int INP_BUFF_DEPTH = 1 << vta_config::UOP_SRC_WIDTH; // 2048 input tiles
constexpr int WGT_BUFF_DEPTH = 1 << vta_config::UOP_WGT_WIDTH; // 1024 weight tiles

// Micro-op masks (derived from vta_config.h – replaces magic 0x7FF / 0x3FF)
constexpr uint32_t DST_MASK = (1u << vta_config::UOP_DST_WIDTH) - 1; // 0x7FF
constexpr uint32_t SRC_MASK = (1u << vta_config::UOP_SRC_WIDTH) - 1; // 0x7FF
constexpr uint32_t WGT_MASK = (1u << vta_config::UOP_WGT_WIDTH) - 1; // 0x3FF

// — Static SRAM arrays (mirror of vta.cc compute() static declarations) ———
// "static" here = file-scope → values persist between calls, like HLS static

// Private to Compute (mirrors vta.cc compute()'s own static declarations)
static uint32_t uop_mem[UOP_BUFF_DEPTH]; // micro-op SRAM
static int32_t acc_mem[ACC_BUFF_DEPTH][VTA_BLOCK_OUT]; // accumulator tiles

// Shared with Load (filled by LoadModule) and Store (read by StoreModule)
static int8_t inp_mem[INP_BUFF_DEPTH][VTA_BLOCK_IN]; // input tiles
static int8_t wgt_mem[WGT_BUFF_DEPTH][VTA_BLOCK_OUT * VTA_BLOCK_IN]; // weight tiles
static int8_t out_mem[ACC_BUFF_DEPTH][VTA_BLOCK_OUT]; // output tiles
```

### Why `int32_t` for `acc_mem` and `int8_t` for `inp/wgt/out`?

In `vta.cc`: `acc_T = ap_int<VTA_ACC_WIDTH>` where `VTA_ACC_WIDTH = 32` → `int32_t`.

`inp_T = ap_int<VTA_INP_WIDTH>` where `VTA_INP_WIDTH = 8` → `int8_t`.

`wgt_T = ap_int<VTA_WGT_WIDTH>` where `VTA_WGT_WIDTH = 8` → `int8_t`.

`out_T = ap_int<VTA_OUT_WIDTH>` where `VTA_OUT_WIDTH = 8` → `int8_t`.

## 5. GEMM Implementation — Using Only Real Constants

### 5.1 Exact field mapping: HLS VTAGemInsn → Instruction\* getters

| vta.cc field        | SystemC getter                     |
|---------------------|------------------------------------|
| insn.iter_out       | current->get_outer_loop_iter()     |
| insn.iter_in        | current->get_inner_loop_iter()     |
| insn.uop_bgn        | current->get_range_0()             |
| insn.uop_end        | current->get_range_1()             |
| insn.dst_factor_out | current->get_gemm_outer_loop_acc() |
| insn.src_factor_out | current->get_gemm_outer_loop_inp() |
| insn.wgt_factor_out | current->get_gemm_outer_loop_wgt() |
| insn.dst_factor_in  | current->get_gemm_inner_loop_acc() |
| insn.src_factor_in  | current->get_gemm_inner_loop_inp() |
| insn.wgt_factor_in  | current->get_gemm_inner_loop_wgt() |
| insn.reset_reg      | current->get_reset_out()           |

### 5.2 GEMM code (no magic numbers — all constants named and explained)

```
// — GEMM: inp × wgt → acc → out —————
// Mirrors: EXE_OUT_LOOP > EXE_IN_LOOP > READ_GEMM_UOP in vta.cc gemm()
// No new variables – uses only Instruction* getters and the static arrays above.

int dst_offset_out = 0; // mirrors vta.cc: acc_idx_T dst_offset_out = 0
int src_offset_out = 0; // mirrors vta.cc: inp_idx_T src_offset_out = 0
int wgt_offset_out = 0; // mirrors vta.cc: wgt_idx_T wgt_offset_out = 0

// EXE_OUT_LOOP
for (int it_out = 0; it_out < current->get_outer_loop_iter(); it_out++) {

    int dst_offset_in = dst_offset_out; // vta.cc: acc_idx_T dst_offset_in = dst_offset_out
    int src_offset_in = src_offset_out;
    int wgt_offset_in = wgt_offset_out;

    // EXE_IN_LOOP
    for (int it_in = 0; it_in < current->get_inner_loop_iter(); it_in++) {

        // READ_GEMM_UOP
        for (int upc = current->get_range_0(); upc < current->get_range_1(); upc++) {

            uint32_t uop = uop_mem[upc]; // vta.cc: uop_T uop = uop_mem[upc]
```

```

// Decode indices – replaces vta.cc's uop.range() using our named masks
int dst_idx = (int)((uop >> 0) & DST_MASK) + dst_offset_in;
int src_idx = (int)((uop >> vta_config::UOP_DST_WIDTH) & SRC_MASK) + src_offset_in;
int wgt_idx = (int)((uop >> (vta_config::UOP_DST_WIDTH + vta_config::UOP_SRC_WIDTH)) & WGT_M

// Inner MAC loop – vta.cc loops over VTA_BATCH, VTA_BLOCK_OUT, VTA_BLOCK_IN
for (int oc = 0; oc < VTA_BLOCK_OUT; oc++) { // output channel loop

    int32_t accum = acc_mem[dst_idx][oc]; // read accumulator tile element
    int32_t tmp = 0;

    for (int ic = 0; ic < VTA_BLOCK_IN; ic++) { // input channel dot-product
        int8_t w = wgt_mem[wgt_idx][oc * VTA_BLOCK_IN + ic]; // weight[oc][ic]
        int8_t x = inp_mem[src_idx][ic]; // input[ic]
        tmp += (int32_t)(x * w); // multiply-accumulate
    }

    accum += tmp;

    // vta.cc: a_tensor[b][oc] = insn.reset_reg ? (acc_T) 0 : accum
    acc_mem[dst_idx][oc] = current->get_reset_out() ? 0 : accum;

    // vta.cc: o_tensor[b][oc] = (out_T) accum.range(VTA_OUT_WIDTH-1, 0)
    // = truncate 32-bit accum to 8-bit: keep lowest 8 bits
    out_mem[dst_idx][oc] = (int8_t)(accum & 0xFF);
}
}

// vta.cc: dst_offset_in += insn.dst_factor_in; etc.
dst_offset_in += current->get_gemm_inner_loop_acc();
src_offset_in += current->get_gemm_inner_loop_inp();
wgt_offset_in += current->get_gemm_inner_loop_wgt();
}

// vta.cc: dst_offset_out += insn.dst_factor_out; etc.
dst_offset_out += current->get_gemm_outer_loop_acc();
src_offset_out += current->get_gemm_outer_loop_inp();
wgt_offset_out += current->get_gemm_outer_loop_wgt();
}

```

### Why & 0xFF for out\_mem?

In vta.cc: `o_tensor[b][oc] = (out_T) accum.range(VTA_OUT_WIDTH-1, 0)`

`out_T = ap_int<VTA_OUT_WIDTH>` and `VTA_OUT_WIDTH = 8`. So `.range(7, 0)` keeps only the lowest 8 bits. In C++ this is `accum & 0xFF`, then cast to `int8_t`. The value `0xFF = (1<<8)-1` — same derivation as our masks, just for 8 bits.

## 6. ALU Implementation — Using Only Real Constants

### 6.1 Exact field mapping: HLS VTAAluInsn → Instruction\* getters

| vta.cc field        | SystemC getter                    |
|---------------------|-----------------------------------|
| insn.iter_out       | current->get_outer_loop_iter()    |
| insn.iter_in        | current->get_inner_loop_iter()    |
| insn.uop_bgn        | current->get_range_0()            |
| insn.uop_end        | current->get_range_1()            |
| insn.dst_factor_out | current->get_alu_outer_loop_dst() |
| insn.src_factor_out | current->get_alu_outer_loop_src() |
| insn.dst_factor_in  | current->get_alu_inner_loop_dst() |
| insn.src_factor_in  | current->get_alu_inner_loop_src() |

#### What about insn.alu\_opcode and insn.use\_imm / insn.imm?

In `vta.cc` these are fields of `VTAAluInsn`. The `Instruction` class in this project does not expose a getter for `alu_opcode` or `use_imm`. However, `vta_config.h` already defines `enum class AluOpcode { MIN=0, MAX=1, ADD=2, SHR=3, MUL=4 }`. We use `current->get_name()` to determine the operation, since the CSV encodes the ALU variant in the instruction name.

### 6.2 ALU micro-op layout

```
// ALU micro-op (32 bits):
// Bit 31      22  21      11  10      0
//
// | (unused) | src_base | dst_base |
// |          | (11 bits) | (11 bits) |
//
// Same bit positions as GEMM, but no wgt field.

int dst_base = (int)((uop >> 0) & DST_MASK);
int src_base = (int)((uop >> vta_config::UOP_DST_WIDTH) & SRC_MASK);
// (wgt bits [31:22] exist but are not used in ALU)
```

### 6.3 ALU code

```
// — ALU: element-wise operation on accumulator tiles —————
// Mirrors: EXE_OUT_LOOP > EXE_IN_LOOP > READ_ALU_UOP in vta.cc alu()
// Difference from GEMM: no wgt_offset, no weight memory, two acc tiles as operands.

std::string name = current->get_name(); // name encodes which ALU op (MAX, MIN, ADD, SHR)
```

```

int dst_offset_out = 0; // vta.cc: acc_idx_T dst_offset_out = 0
int src_offset_out = 0; // vta.cc: inp_idx_T src_offset_out = 0

// EXE_OUT_LOOP
for (int it_out = 0; it_out < current->get_outer_loop_iter(); it_out++) {

    int dst_offset_in = dst_offset_out;
    int src_offset_in = src_offset_out;

    // EXE_IN_LOOP
    for (int it_in = 0; it_in < current->get_inner_loop_iter(); it_in++) {

        // READ_ALU_UOP
        for (int upc = current->get_range_0(); upc < current->get_range_1(); upc++) {

            uint32_t uop = uop_mem[upc];

            // Decode - ALU only uses dst and src, not wgt
            int dst_idx = (int)((uop >> 0) & DST_MASK) + dst_offset_in;
            int src_idx = (int)((uop >> vta_config::UOP_DST_WIDTH) & SRC_MASK) + src_offset_in;

            for (int oc = 0; oc < VTA_BLOCK_OUT; oc++) {

                // vta.cc: acc_T src_0 = dst_tensor[i][b] (first operand from dst tile)
                int32_t src_0 = acc_mem[dst_idx][oc];
                // vta.cc: acc_T src_1 = insn.use_imm ? (acc_T)insn.imm : src_tensor[i][b]
                // (we read from acc_mem[src_idx]; for ReLU this tile holds 0s)
                int32_t src_1 = acc_mem[src_idx][oc];

                int32_t result;

                // vta.cc checks insn.alu_opcode - we match using instruction name
                // (vta_config::AluOpcode::MAX=1, MIN=0, ADD=2, SHR=3)
                if (name.find("MAX") != std::string::npos) {
                    result = (src_0 > src_1) ? src_0 : src_1; // MAX → ReLU when src_1=0
                } else if (name.find("MIN") != std::string::npos) {
                    result = (src_0 < src_1) ? src_0 : src_1; // MIN
                } else if (name.find("ADD") != std::string::npos) {
                    result = src_0 + src_1; // ADD → residual connection
                } else {
                    result = src_0 >> src_1; // SHR → fixed-point scaling
                }

                // vta.cc: write back to dst tensor and output buffer
                acc_mem[dst_idx][oc] = result;
                out_mem[dst_idx][oc] = (int8_t)(result & 0xFF);
            }
        }

        dst_offset_in += current->get_alu_inner_loop_dst();
        src_offset_in += current->get_alu_inner_loop_src();
    }

    dst_offset_out += current->get_alu_outer_loop_dst();
}

```



```
src_offset_out += current->get_alu_outer_loop_src();  
}
```

## 7. Complete dependencies\_received() — Final Version

```

void ComputeModule::dependencies_received() {
    std::string name = current->get_name();

    // — GEMM —————
    if (name == "GEMM") {
        int dst_offset_out = 0, src_offset_out = 0, wgt_offset_out = 0;

        for (int it_out = 0; it_out < current->get_outer_loop_iter(); it_out++) {
            int dst_offset_in = dst_offset_out;
            int src_offset_in = src_offset_out;
            int wgt_offset_in = wgt_offset_out;

            for (int it_in = 0; it_in < current->get_inner_loop_iter(); it_in++) {
                for (int upc = current->get_range_0(); upc < current->get_range_1(); upc++) {
                    uint32_t uop = uop_mem[upc];

                    int dst_idx = (int)((uop >> 0) & DST_MASK) + dst_offset_in;
                    int src_idx = (int)((uop >> vta_config::UOP_DST_WIDTH) & SRC_MASK) + src_offset_in;
                    int wgt_idx = (int)((uop >> (vta_config::UOP_DST_WIDTH + vta_config::UOP_SRC_WIDTH))

                    for (int oc = 0; oc < VTA_BLOCK_OUT; oc++) {
                        int32_t accum = acc_mem[dst_idx][oc];
                        int32_t tmp = 0;
                        for (int ic = 0; ic < VTA_BLOCK_IN; ic++) {
                            int8_t w = wgt_mem[wgt_idx][oc * VTA_BLOCK_IN + ic];
                            int8_t x = inp_mem[src_idx][ic];
                            tmp += (int32_t)(x * w);
                        }
                        accum += tmp;
                        acc_mem[dst_idx][oc] = current->get_reset_out() ? 0 : accum;
                        out_mem[dst_idx][oc] = (int8_t)(accum & 0xFF);
                    }
                }
                dst_offset_in += current->get_gemm_inner_loop_acc();
                src_offset_in += current->get_gemm_inner_loop_inp();
                wgt_offset_in += current->get_gemm_inner_loop_wgt();
            }
            dst_offset_out += current->get_gemm_outer_loop_acc();
            src_offset_out += current->get_gemm_outer_loop_inp();
            wgt_offset_out += current->get_gemm_outer_loop_wgt();
        }
    }

    // — ALU (all variants: MAX/MIN/ADD/SHR — catches everything that is not —
    //      GEMM, LOAD ACC, NOP, or FINISH, exactly as latency() does) —
    else if (name != "LOAD ACC" && name != "NOP-COMPUTE-STAGE" && name != "FINISH") {

        int dst_offset_out = 0, src_offset_out = 0;

        for (int it_out = 0; it_out < current->get_outer_loop_iter(); it_out++) {

```

```

int dst_offset_in = dst_offset_out;
int src_offset_in = src_offset_out;

for (int it_in = 0; it_in < current->get_inner_loop_iter(); it_in++) {
    for (int upc = current->get_range_0(); upc < current->get_range_1(); upc++) {
        uint32_t uop = uop_mem[upc];

        int dst_idx = (int)((uop >> 0) & DST_MASK) + dst_offset_in;
        int src_idx = (int)((uop >> vta_config::UOP_DST_WIDTH) & SRC_MASK) + src_offset_in;

        for (int oc = 0; oc < VTA_BLOCK_OUT; oc++) {
            int32_t src_0 = acc_mem[dst_idx][oc];
            int32_t src_1 = acc_mem[src_idx][oc];
            int32_t result;

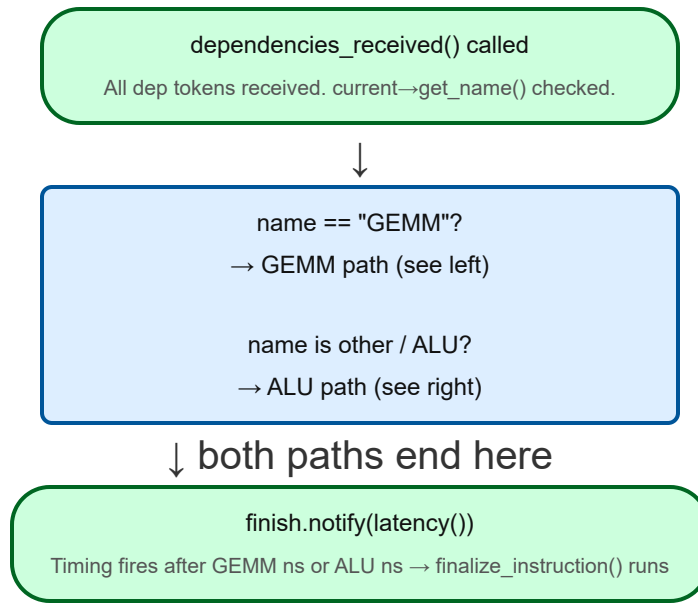
            if (name.find("MAX") != std::string::npos) result = (src_0 > src_1) ? src_0;
            else if (name.find("MIN") != std::string::npos) result = (src_0 < src_1) ? src_0;
            else if (name.find("ADD") != std::string::npos) result = src_0 + src_1;
            else result = src_0 >> src_1;

            acc_mem[dst_idx][oc] = result;
            out_mem[dst_idx][oc] = (int8_t)(result & 0xFF);
        }
        dst_offset_in += current->get_alu_inner_loop_dst();
        src_offset_in += current->get_alu_inner_loop_src();
    }
    dst_offset_out += current->get_alu_outer_loop_dst();
    src_offset_out += current->get_alu_outer_loop_src();
}

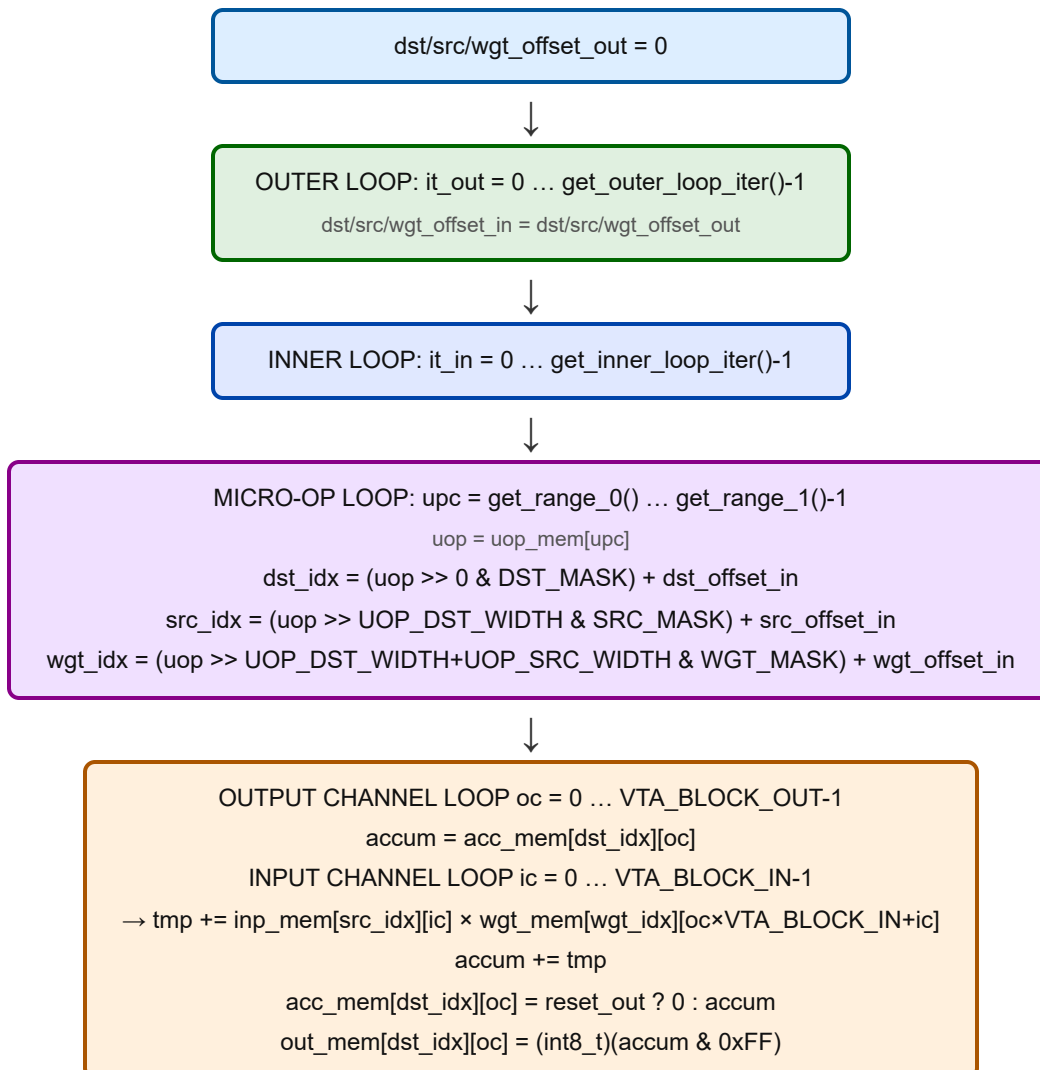
// — LOAD ACC / NOP / FINISH: no computation — timing only —————
finish.notify(latency());
}

```

## 8. Dataflow Flowchart



### GEMM path detail:





```
After inner: dst/src/wgt_offset_in += get_gemm_inner_loop_acc(inp/wgt())
After outer: dst/src/wgt_offset_out += get_gemm_outer_loop_acc(inp/wgt())
```

9. Everything in One Place — What Goes Where

| File               | What you add                                                                           | Where in the file                                                               |
|--------------------|----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| compute_module.cpp | 6 constexpr constants (VTA_BATCH, VTA_BLOCK_IN, VTA_BLOCK_OUT, plus buffer depths)     | After the #include lines, before any function                                   |
| compute_module.cpp | 3 mask constants (DST_MASK, SRC_MASK, WGT_MASK) — derived from vta_config::UOP_*_WIDTH | Same location, after the buffer depth constants                                 |
| compute_module.cpp | 5 static arrays (uop_mem, acc_mem, inp_mem, wgt_mem, out_mem)                          | Same location, mirrors vta.cc compute()'s static declarations                   |
| compute_module.cpp | GEMM + ALU loop code inside dependencies_received()                                    | Replace the existing single-line body; keep finish.notify(latency()) at the end |
| compute_module.h   | Nothing — professor confirmed everything needed is already declared                    | —                                                                               |